

Introduction to Java

1. History of Java

- Java was developed by **James Gosling** at **Sun Microsystems** in **1995**
- Initially called **Oak** (later renamed Java)
- Designed for **embedded systems**, later used for web & software development
- Now maintained by **Oracle Corporation**

👉 **Main Goal:** Platform Independent Programming (*Write Once, Run Anywhere*)

2. Java Buzzwords (Features of Java)

These are the **special characteristics** of Java:

- **Simple** → Easy to learn (similar to C/C++)
- **Object-Oriented** → Based on classes & objects
- **Platform Independent** → Runs on any OS using JVM
- **Secure** → No pointer manipulation, strong memory control
- **Robust** → Strong error handling & memory management
- **Multithreaded** → Supports multiple tasks simultaneously
- **Portable** → Same program runs everywhere
- **Distributed** → Supports network-based applications
- **Dynamic** → Can adapt at runtime

3. Basics of Java Programming

✅ Structure of Java Program

```
class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

Difference Between Procedural and Object-Oriented Programming

◆ Basis	◆ Procedural Programming	◆ Object-Oriented Programming (OOP)
Approach	Function-based	Object-based
Focus	Procedures / Functions	Objects & Classes
Data Security	Less secure (no data hiding)	More secure (Encapsulation)
Reusability	Limited	High (Inheritance)
Structure	Top-down approach	Bottom-up approach

◆ Basis	◆ Procedural Programming	◆ Object-Oriented Programming (OOP)
Data Handling	Data & functions are separate	Data & functions are combined
Real-world Mapping	Not easy	Easy (real-world modeling)
Maintenance	Difficult for large programs	Easy to maintain
Examples	C, Pascal	Java, C++, Python

◆ Short Definition

- **Procedural Programming:**

A programming paradigm based on **functions and procedures**.

- **Object-Oriented Programming:**

A paradigm based on **objects, classes, and real-world entities**.

🔥 Key Point for Exam

👉 Procedural = **Function oriented**

👉 OOP = **Object oriented (more secure + reusable)**

■ Need for OOPs Paradigm

OOP (Object-Oriented Programming) is needed to overcome limitations of procedural programming:

- ◆ **Code Reusability** → Using inheritance, same code can be reused
- ◆ **Data Security** → Encapsulation hides sensitive data
- ◆ **Real-World Modeling** → Represents real-life objects easily
- ◆ **Easy Maintenance** → Modular structure makes debugging easier
- ◆ **Scalability** → Suitable for large and complex programs
- ◆ **Reduced Complexity** → Breaks problem into smaller objects

■ Features of OOPs

◆ 1. Class

- Blueprint of objects

◆ 2. Object

- Instance of class

◆ 3. Encapsulation

- Wrapping data + methods into one unit
- Provides data hiding

◆ 4. Inheritance

- One class acquires properties of another

◆ 5. Polymorphism

- One method, many forms
 - Overloading (compile-time)
 - Overriding (runtime)

◆ 6. Abstraction

- Hiding internal details, showing only essential features

Advantages of OOPs

- ✓ **Reusability** → Code reuse using inheritance
- ✓ **Security** → Data hiding using encapsulation
- ✓ **Flexibility** → Easy to modify and extend
- ✓ **Modularity** → Program divided into small parts (classes)
- ✓ **Maintainability** → Easy to debug and update
- ✓ **Real-world approach** → Easier understanding
- ✓ **Reduced Redundancy** → Less code duplication

Very Short Exam Summary

👉 **Need:** Reusability, Security, Real-world modeling

👉 **Features:** Class, Object, Encapsulation, Inheritance, Polymorphism, Abstraction

👉 **Advantages:** Reusable, Secure, Easy to maintain

JDK, JRE and JVM

1. JVM (Java Virtual Machine)

- JVM is a **virtual machine** that runs Java bytecode
- It converts **bytecode** → **machine code**
- Makes Java **platform independent**

✓ Functions:

- Executes program
- Memory management
- Garbage collection

👉 Key Point:

JVM = Runs Java Program

2. JRE (Java Runtime Environment)

- JRE provides **environment to run Java programs**
- It includes:

- JVM
- Libraries (API classes)

👉 Key Point:

JRE = JVM + Libraries

3. JDK (Java Development Kit)

- JDK is used to **develop Java programs**
- It includes:
 - JRE
 - Development tools (compiler, debugger)

✓ Important Tools:

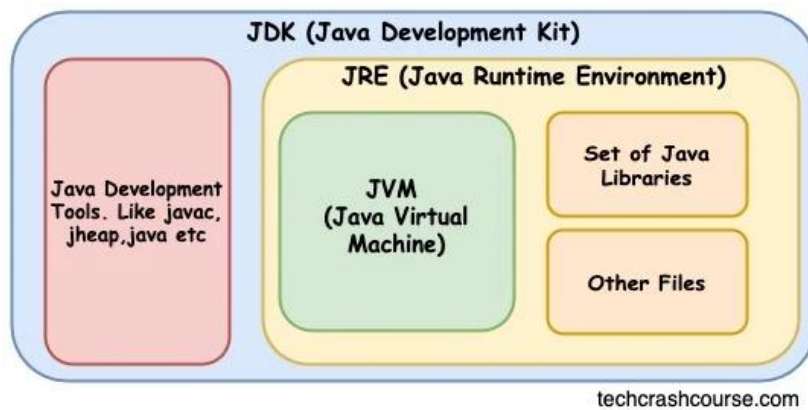
- javac → Compiler
- java → Run program

👉 Key Point:

JDK = JRE + Development Tools

🔥 Difference Table

Feature	JVM	JRE	JDK
Full Form	Java Virtual Machine	Java Runtime Environment	Java Development Kit
Purpose	Run bytecode	Run Java programs	Develop + Run programs
Contains	Execution engine	JVM + Libraries	JRE + Tools
User	System	End user	Developer

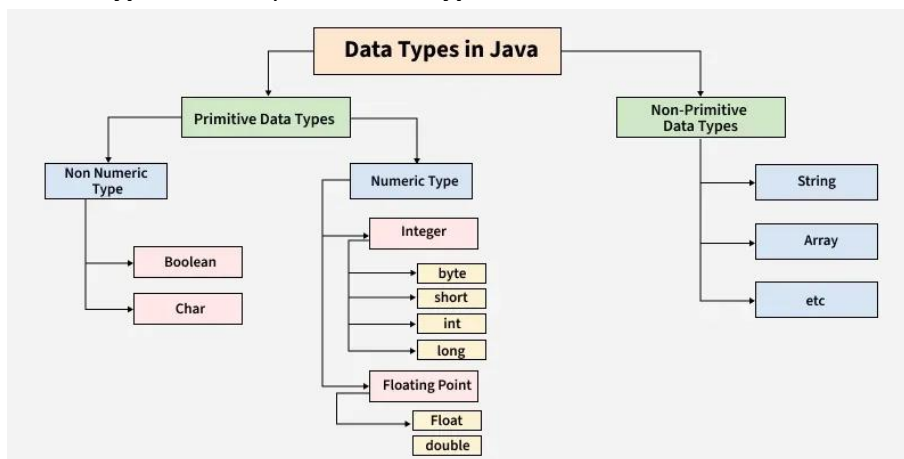


techcrashcourse.com

Data Types in Java

◆ Definition (2–3 Marks)

A **data type** in Java specifies the **type of value** a variable can store.



Type	Description	Default	Size	Example Literals
boolean	true or false	false	1 bit	true, false
byte	twos complement integer	0	8 bits	(none)
char	Unicode character	\u0000	16 bits	'a', '\u0041', '\101', '\\', '\'', '\n', '\b'
short	twos complement integer	0	16 bits	(none)
int	twos complement integer	0	32 bits	-2, -1, 0, 1, 2
long	twos complement integer	0	64 bits	-2L, -1L, 0L, 1L, 2L
float	IEEE 754 floating point	0.0	32 bits	1.23e100f, -1.23e-100f, .3f, 3.14F
double	IEEE 754 floating point	0.0	64 bits	1.23456e300d, -1.23456e-300d, 1e1d

Variables in Java

◆ Definition (2–3 Marks)

A **variable** is a **named memory location** used to store data values in a program.

Variable is used to Store Data



Operators in Java

◆ Definition (2–3 Marks)

An **operator** is a symbol used to **perform operations on variables and values**.

👉 Example:

```
int c = a + b;
```

◆ Types of Operators in Java

◆ 1. Arithmetic Operators

Used for mathematical operations

Operator	Meaning	Example
+	Addition	$a + b$
-	Subtraction	$a - b$
*	Multiplication	$a * b$
/	Division	a / b
%	Modulus	$a \% b$

◆ 2. Relational (Comparison) Operators

Used to compare values

Operator	Meaning
==	Equal to
!=	Not equal
>	Greater than
<	Less than
>=	Greater than equal
<=	Less than equal

👉 Output is always **true or false**

◆ 3. Logical Operators

Used with conditions

Operator Meaning

&&	AND
!	NOT

◆ 4. Assignment Operators

Used to assign values

Operator Example

=	$a = 10$
+=	$a += 5$
-=	$a -= 2$
*=	$a *= 3$
/=	$a /= 2$

◆ 5. Increment & Decrement Operators

Operator Meaning

++	Increase by 1
--	Decrease by 1

👉 Example:

```
int a = 5;
```

```
a++; // a = 6
```

◆ 6. Bitwise Operators (Short Note)

- Work on binary values
- &, |, ^, ~, <<, >>

Control Structures in Java (Selection Statements)

◆ Definition (2–3 Marks)

Control structures are used to **control the flow of execution** of a program.

👉 **Selection statements** choose one block of code based on a condition.

◆ Types of Selection Statements

◆ 1. if Statement

- Executes block **only if condition is true**

```
if(condition) {  
    // code  
}
```

👉 Example:

```
if(a > 10) {  
    System.out.println("Greater");  
}
```

◆ 2. if-else Statement

- Executes one block if true, another if false

```
if(condition) {  
    // true block  
} else {  
    // false block  
}
```

◆ 3. if-else-if Ladder

- Used for **multiple conditions**

```
if(condition1) {  
    // code  
} else if(condition2) {  
    // code  
} else {  
    // default  
}
```

◆ 4. Nested if

- if inside another if

```
if(condition1) {  
    if(condition2) {  
        // code  
    }  
}
```

◆ 5. switch Statement

- Used for **multiple choices**

```
switch(expression) {  
    case 1:  
        // code  
        break;  
    case 2:  
        // code  
        break;  
    default:  
        // code  
}
```

◆ Key Points (Exam Focus)

- if → single condition
 - if-else → two conditions
 - if-else-if → multiple conditions
 - switch → multiple options (faster than if ladder)
 - break is used in switch to stop execution
-

Looping Statements in Java

◆ Definition (2–3 Marks)

Looping statements are used to **execute a block of code repeatedly** until a condition is satisfied.

◆ Types of Loops in Java

◆ 1. for Loop

- Used when **number of iterations is known**

```
for(initialization; condition;
increment/decrement) {
    // code
}
```

👉 Example:

```
for(int i = 1; i <= 5; i++) {
    System.out.println(i);
}
```

◆ 2. while Loop

- Used when **condition is checked first**

```
while(condition) {
    // code
}
```

👉 Example:

```
int i = 1;
while(i <= 5) {
    System.out.println(i);
    i++;
}
```

◆ 3. do-while Loop

- Executes **at least once** (condition checked later)

```
do {
    // code
} while(condition);
```

👉 Example:

```
int i = 1;
do {
    System.out.println(i);
    i++;
} while(i <= 5);
```

◆ Key Differences (Important)

Loop	Condition Check	Execution
for	Before loop	0 or more times
while	Before loop	0 or more times
do-while	After loop	At least 1 time

◆ Loop Control Statements (Short Note)

- break → exit loop immediately
- continue → skip current iteration

🔥 Key Points (Exam Focus)

- Use for when iterations are fixed
 - Use while when condition-based
 - do-while always runs at least once
 - Infinite loop possible if condition never false
-

Java Methods

◆ Definition (2–3 Marks)

A **method** in Java is a **block of code that performs a specific task** and is executed when called.

◆ Syntax of Method

```
returnType  
methodName(parameters) {  
    // code  
    return value;  
}
```

👉 Example:

```
int add(int a, int b) {  
    return a + b;  
}
```

◆ Types of Methods

◆ 1. Predefined Methods

- Already defined in Java libraries

👉 Example:

```
System.out.println()
```

◆ 2. User-defined Methods

- Created by programmer

```
void show() {  
    System.out.println("Hello");  
}
```

◆ Method Calling

```
show(); // calling method
```

◆ Method with Parameters

```
void display(int x) {  
    System.out.println(x);  
}
```

◆ Method with Return Value

```
int square(int n) {  
    return n * n;  
}
```

◆ Advantages of Methods

-  Code Reusability
-  Reduces repetition
-  Improves readability
-  Easy debugging

◆ Important Concepts

- **Method Overloading** → Same name, different parameters
- **Static Method** → Can be called without object
- **Void Method** → Does not return value

Key Points (Exam Focus)

- Method = **function in Java**
- Must be **called to execute**
- Can have **parameters and return value**
- **main()** is the **starting method**

■ Compilation and Execution of a Simple Java Program

◆ Definition (2–3 Marks)

- **Compilation:** Converting Java source code (.java) into **bytecode (.class)** using compiler.
- **Execution:** Running the bytecode using **JVM** to produce output.

◆ Steps to Compile and Run Java Program

◆ 1. Write the Program

- Create a file with .java extension

👉 Example: Hello.java

```
class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

◆ 2. Compilation

👉 Command:

javac Hello.java

- Uses **Java Compiler (javac)**
- Generates → Hello.class (bytecode)

◆ 3. Execution (Run Program)

👉 Command:

java Hello

- JVM loads and executes bytecode
- Output is displayed on screen

◆ Flow of Execution

Source Code (.java)

↓ javac

Bytecode (.class)

↓ JVM

Output

◆ Key Points (Exam Focus)

- File name must match **class name**
- main() is the **entry point**
- Compilation error → occurs at compile time
- Runtime error → occurs during execution
- Java is **platform independent** due to JVM

👉 .java → javac → .class → JVM →

Output

👉 Compile = **convert code**

👉 Execute = **run program**